IIIF Technical and API Study Guide

Project-focused English and Chinese edition / 项目导向英中双语版

Boheng Shang

2026-09-14

Document scope and sources

This study guide is based on the current stable IIIF specifications relevant to the semester project. It is a project-focused explanation rather than a replacement for the normative specifications. The downloaded English specifications are preserved separately in `official-originals`. Where this guide and a specification differ, the official English specification is authoritative.

Official sources consulted on 14 September 2026:

- How IIIF Works: https://iiif.io/get-started/how-iiif-works/
- API Specifications: https://iiif.io/api/
- Presentation API 3.0.0: https://iiif.io/api/presentation/3.0/
- Image API 3.0.0: https://iiif.io/api/image/3.0/
- Change Discovery API 1.0.0: https://iiif.io/api/discovery/1.0/
- IIIF Registry machine entry point: https://registry.iiif.io/index.json

Part I —English study version

1. What IIIF is

IIIF stands for the International Image Interoperability Framework. It is a family of open standards used by libraries, museums, archives, and other organizations to publish images, audio, video, and structured digital objects consistently on the Web.

IIIF does not place every institution' s data in one central database. Each provider continues to host its own resources. IIIF defines common machine-readable structures and request patterns so that clients can work with resources from different institutions in a similar way.

The three APIs most relevant to this project have distinct roles:

Component	Main role	What our pipeline obtains
Change Discovery API	Announces available, new, changed, moved, or deleted IIIF resources	Manifest and Collection references, activity types, and timestamps when available
Presentation API	Describes the structure and presentation of a digital object	Manifest metadata, page order, rights, provider information, and links to content
Image API	Delivers image pixels and technical image information	Image service information and requested image derivatives

The IIIF Registry is a directory of registered Change Discovery streams. It is useful for a provider census, but registration is voluntary and therefore it is not a complete census of all IIIF providers.

2. How the main IIIF resources relate

A simplified Presentation API 3 structure is:

```
Collection
└── Manifest
    └── Canvas (one view, page, or time span)
        └── AnnotationPage
            └── Annotation
                └── Content Resource, often an image
                    └── optional Image API service
```

- A **Collection** groups Manifests or child Collections.
- A **Manifest** is the main description of one digital object, such as a book, map, painting, or photograph.

- A **Canvas** is an abstract space or time frame on which content is placed. In a digitized book, one Canvas commonly represents one page view.
- An **AnnotationPage** groups Annotations.
- An **Annotation** connects a body, such as an image, to a target, usually a Canvas.
- A **Content Resource** is the image, audio, video, or text associated with the object.
- An **Image API service** may be linked from an image resource and allows standard image requests.

Clients should normally follow the identifiers and links supplied by the retrieved JSON. They should not guess provider-specific paths. The Image API is the main exception because its defined URI template is intentionally used to construct image requests.

3. Presentation API 3.0 for Manifest harvesting

The Presentation API represents resources in JSON-LD. For a Manifest, the most important fields for the first pipeline version include:

Field	Meaning in the specification	Use in this project
`id`	URI identifying the resource	Stable Manifest identifier and provenance
`type`	Resource class, normally `Manifest`	Basic structural validation
`label`	Human-readable name in a language map	Title for inspection and downstream display
`summary`	Short human-readable description	Optional descriptive context
`metadata`	Ordered label-value pairs for display	Raw provider metadata; labels vary across providers
`rights`	Rights or license URI	Raw rights value for normalization and filtering
`requiredStatement`	Required attribution or use statement	Preserve when present; do not confuse it with `rights`
`provider`	Agent or institution associated with the resource	Provider provenance
`homepage`	Human-facing page about the object	Link back to the provider’ s website
`seeAlso`	External machine-readable description	Possible source of richer domain metadata
`items`	Ordered Canvases	Page or view sequence and route toward image resources

The `metadata` field is intended mainly for human-readable display. Providers may use different labels, languages, and value formats for the same concept. A pipeline therefore cannot assume that date or place is always stored under one fixed key. Raw values should be preserved before normalization.

The Presentation API deliberately does not define a complete semantic metadata model for advanced searching. When a Manifest links to richer descriptions through `seeAlso`, those linked resources may provide better structured metadata, but support should be added only when required by the project scope.

A basic structural check for a Presentation API 3 Manifest should at least verify that the response is valid JSON, that `type` is `Manifest`, and that the required top-level properties expected by the specification are present. Validation failure should be recorded rather than silently discarded.

4. Change Discovery API 1.0

The Change Discovery API provides a machine-to-machine method for discovering IIIF resources. It uses the W3C Activity Streams model.

A provider publishes an `OrderedCollection`. The collection points to `OrderedCollectionPage` resources. Each page contains ordered activities. An activity normally refers to a IIIF resource through `object.id` and identifies the resource type through `object.type`.

Illustrative activity:

```
{
  "type": "Update",
  "object": {
    "id": "https://example.org/iiif/object-1/manifest",
    "type": "Manifest"
  },
  "endTime": "2026-09-10T12:00:00Z"
}
```

The activity is a reference to a resource and a statement about its state or change. It is not the Manifest itself. After discovering the URL, the harvester must request the current Manifest separately.

The specification defines three conformance levels:

Level	Information available	Consequence for a harvester
Level 0	Unique references to resources	Supports basic discovery, but gives limited incremental information

Level	Information available	Consequence for a harvester
Level 1	Timestamps and ordering	Allows a client to stop after reaching already processed changes
Level 2	Explicit activity types such as `Create`, `Update`, `Delete`, and `Move`	Supports more complete synchronization of additions, changes, deletions, and moves

The discovery module should:

1. Read the Ordered Collection.
2. Follow its page links in the order required by the harvesting strategy.
3. Read every relevant activity on each page.
4. retain the activity type and time when available.
5. extract the URL only when the object is a relevant resource, especially a Manifest.
6. deduplicate references without losing provenance or the most recent activity.
7. store a checkpoint for later incremental runs.

The module aggregates Manifest references; it does not merge multiple Manifest JSON documents into one Manifest.

5. IIIF Registry

The machine-readable Registry entry point is:

```
https://registry.iiif.io/index.json
```

It is an `OrderedCollection` that points to Registry pages. Registered activities lead to provider discovery streams. For this project, the Registry can seed the provider census:

```
Registry index
→ Registry page
→ registered provider stream
→ provider stream pages
→ Manifest references
→ current Manifest JSON
```

The Registry website and the Registry JSON serve different purposes. The website is for people; `index.json` is the machine entry point. Similarly, the Change Discovery specification page explains the standard but is not a dataset to harvest.

6. Image API 3.0

The Image API returns images in response to standardized URI requests. Its main request template is:

```
{scheme}://{server}{/prefix}/{identifier}/{region}/{size}/{rotation}/{quality}.{format}
```

Example:

```
https://example.org/image-service/abc/full/max/0/default.jpg
```

The parameters are applied conceptually in this order: region, size, rotation, quality, and format. An image service also exposes technical information at:

```
{scheme}://{server}{/prefix}/{identifier}/info.json
```

For this semester project, image downloading should occur only after the Manifest has passed the confirmed rights rule and only when the downstream task actually needs image files. The pipeline should preserve the Image API service URL even when it does not download the image immediately.

7. Project harvesting workflow

The recommended project workflow, expressed using the current machine design, is:

```
Provider storage and selection
→ entry-point access and availability check
→ resource discovery and pagination
→ Manifest retrieval and structural validation
→ rights identification, normalization, and filtering
→ metadata extraction and normalization
→ common-schema validation and export
→ Hafidz's downstream extraction and Time Atlas
```

7.1 Provider storage and selection

Input: the Registry and other supervisor-approved provider sources.

Store the provider name, institution page, discovery endpoint, collection focus, estimated scale, rights observations, status, and last checked time. Select providers through explicit configuration rather than hard-coded branching.

Output: provider records and the entry points selected for the current run.

7.2 Entry-point access check

Input: one provider entry point.

Request it with a timeout and a clear client identifier. Record the final URL after redirection, HTTP status, response content type, access time, and failure category. A successful HTTP response alone does not prove that the result is valid IIIF JSON.

Output: an access result and the response or validated URL for the discovery module.

7.3 Resource discovery

Input: a valid Change Discovery stream or IIIF Collection.

Follow pagination, identify relevant resource references, deduplicate URLs, and preserve the discovery origin, activity type, and timestamp. Change Discovery and Presentation Collections require different parsers behind a common output interface.

Output: a queue or table of Manifest URLs with provenance and discovery status.

7.4 Manifest retrieval and validation

Input: a Manifest URL.

Retrieve the current JSON, record the HTTP result, parse it, detect its Presentation API version where possible, and verify the basic structure. Cache or hash the raw response to avoid unnecessary duplicate work and to support later comparisons.

Output: the raw Manifest plus validation status, or an explicit failure record.

7.5 Rights processing

Input: a structurally usable Manifest.

Preserve the raw rights URI or statement. Map known statements to normalized categories and apply only the allowlist confirmed by the supervisors. Use explicit states such as `allowed`, `excluded`, `unknown`, and `pending_policy`. Record the reason for every exclusion.

Output: a rights decision and an auditable explanation.

7.6 Metadata extraction and normalization

Input: a rights-approved Manifest.

Extract at least identifier, date, place, rights, source institution, Manifest URL, and discovery origin. Keep raw and normalized values separately. Distinguish the holding institution from the historical or depicted place. Represent missing or ambiguous data explicitly.

Output: a provisional normalized record.

7.7 Common-schema validation and export

Input: a provisional normalized record.

Map it to the versioned team schema. Validate required fields, types, missing-value rules, and provenance. Export JSONL or the handoff format confirmed with Hafidz and Kaile.

Output: schema-conformant records and a separate validation-error report.

8. Minimal record for the first prototype

The final schema must be confirmed with the team. A provisional record for discussion could look like this:

```
{
  "identifier": "https://example.org/iiif/object-1/manifest",
  "date": {
    "raw": "circa 1890",
    "normalized": "1890",
    "status": "approximate"
  },
  "place": {
    "raw": "Lausanne",
    "normalized": "Lausanne, Switzerland",
    "status": "normalized"
  },
  "rights": {
    "raw": "https://rightsstatements.org/vocab/NoC-US/1.0/",
    "category": "no_known_copyright",
    "decision": "pending_policy"
  },
  "source": {
    "provider": "Example Institution",
    "manifest_url": "https://example.org/iiif/object-1/manifest",
    "discovery_endpoint": "https://example.org/activity-stream"
  },
```

```
  "processing": {
    "retrieved_at": "2026-09-14T12:00:00Z",
    "status": "normalized",
    "schema_version": "provisional-0.1"
  }
}
```


This is an implementation proposal for discussion, not a supervisor-approved schema or rights decision.

9. Reliability principles

The first target should be a small but complete path through every module. Once 10–100 real Manifests can move from discovery to export, improve the system with retries, rate limiting, checkpoints, deduplication, caching, statistics, and reproducible configuration.

Every record should remain traceable to its provider, discovery endpoint, and Manifest URL. Missing values and failures should remain visible. Logs should record at least the source, time, processing stage, result, and reason for failure.

10. Short spoken explanation

> IIIF is a set of standards that allows cultural heritage institutions to publish digital objects in a consistent way. The Change Discovery API helps us find Manifest URLs across providers. A Manifest, defined by the Presentation API, describes one object, its page order, metadata, rights, and links to images. If an image service is available, the Image API provides a standard URL syntax for retrieving the required image. In my project, I will census providers, discover and retrieve Manifests, validate them, filter them by rights, normalize identifier, date, place, rights, and source, and export the records in the team’ s common schema for downstream extraction and Time Atlas.

第二部分——中文学习版

1. IIIF 是什么

IIIF 是 International Image Interoperability Framework 的缩写，中文可译为“国际图像互操作框架”。它是一组开放标准，供图书馆、博物馆、档案馆等机构以一致的方式在网络上发布图像、音频、视频和结构化数字对象。

IIIF 不会把所有机构的数据集中存放到一个中央数据库中。每个提供机构仍然保存自己的资源；IIIF 规定共同的机器可读结构和请求方式，使客户端能够用相近的方法处理来自不同机构的数据。

与本项目关系最密切的三个接口分工如下：

组成部分	主要作用	本流水线获得的内容
Change Discovery API（变更发现接口）	公布现有、新增、更新、移动或删除的 IIIF 资源	Manifest 或 Collection 地址，以及可用时的活动类型和时间
Presentation API（展示接口）	描述一个数字对象的结构和展示方式	Manifest 元数据、页面顺序、版权、提供方及内容链接
Image API（图像接口）	提供图像像素和图像技术信息	图像服务信息及按参数生成的图像

IIIF Registry 是已登记变更发现活动流的目录，适合用作提供机构普查的起点。但是登记是自愿的，因此它不能代表所有 IIIF 提供机构的完整名单。

2. IIIF 主要资源之间的关系

Presentation API 3 的简化结构如下：

```
Collection（集合）
└─ Manifest（资源清单文件）
    └─ Canvas（一个视图、页面或时间区段）
        └─ AnnotationPage（标注页）
            └─ Annotation（标注）
                └─ Content Resource（内容资源，通常是图像）
                    └─ 可选的 Image API 服务
```

- **Collection** 用于组织 Manifest 或下级 Collection。
- **Manifest** 是描述一个数字对象的主要文件，例如一本书、一幅地图、一件绘画或一张照片。
- **Canvas** 是放置内容的抽象空间或时间框架。在数字化书籍中，一个 Canvas 通常对应一个页面视图。
- **AnnotationPage** 用于组织 Annotation。
- **Annotation** 将图像等内容与目标连接起来，目标通常是 Canvas。

- **Content Resource** 是与对象关联的图像、音频、视频或文本。
- 图像资源可以连接 **Image API 服务**，以支持标准化图像请求。

客户端通常应沿着 JSON 中提供的标识符和链接继续访问，不能自行猜测提供方的路径。主要例外是 Image API，因为该接口明确规定了用于构造图像请求的地址模板。

3. 用 Presentation API 3.0 采集 Manifest

Presentation API 使用 JSON-LD 表示资源。第一版流水线需要重点关注下列 Manifest 字段：

字段	规范中的含义	在本项目中的用途
`id`	标识资源的 URI	稳定的 Manifest 标识符及来源追踪
`type`	资源类型，通常为 `Manifest`	基本结构校验
`label`	使用语言映射表示的可读名称	人工检查和下游展示所需标题
`summary`	简短的可读说明	可选的描述信息
`metadata`	用于展示的有序“标签—值”列表	保存提供方原始元数据；不同机构的标签可能不同
`rights`	版权或许可 URI	用于规范化和筛选的原始版权值
`requiredStatement`	必须显示的署名或使用声明	存在时应保留，不能与 `rights` 混为一谈
`provider`	与资源相关的机构或主体	记录数据提供方
`homepage`	面向用户的对象网页	返回提供机构原网页的链接
`seeAlso`	外部机器可读描述	可能提供更丰富、结构更明确的领域元数据
`items`	按顺序排列的 Canvas	表示页面或视图顺序，并通向图像资源

`metadata` 字段主要供人阅读和展示。不同机构可能使用不同标签、语言和格式表达同一概念，因此程序不能假设日期或地点总是出现在一个固定字段中。规范化之前必须保留原始值。

Presentation API 并不定义一套供高级检索使用的完整语义元数据模式。当 Manifest 通过 `seeAlso` 连接到更丰富的描述文件时，这些文件可能提供结构更好的元数据；是否支持它们，应根据项目实际范围决定。

对 Presentation API 3 Manifest 进行初步结构校验时，至少应检查响应是否为有效 JSON、`type` 是否为 `Manifest`，以及规范要求的顶层属性是否存在。校验失败应形成明确记录，不能被静默丢弃。

4. Change Discovery API 1.0

Change Discovery API 为发现 IIIF 资源提供机器对机器的统一方法。它采用 W3C Activity Streams（活动流）模型。

提供机构发布一个 `OrderedCollection`（有序集合）。有序集合连接若干 `OrderedCollectionPage`（有序集合页），每页包含按顺序排列的活动。活动一般通过 `object.id` 指向 IIIF 资源，并通过 `object.type` 标明资源类型。

示意活动如下：

```
{
  "type": "Update",
  "object": {
    "id": "https://example.org/iiif/object-1/manifest",
    "type": "Manifest"
  },
  "endTime": "2026-09-10T12:00:00Z"
}
```

活动只是一个资源引用以及对资源状态或变化的说明，它不是 Manifest 本身。流水线发现地址后，仍需单独请求当前 Manifest。

规范规定了三个符合等级：

等级	可用信息	对采集程序的作用
Level 0	不重复的资源引用	支持基本发现，但增量更新信息有限
Level 1	时间戳和顺序	客户端遇到已经处理的变更后可以停止继续读取
Level 2	`Create`、`Update`、`Delete`、`Move` 等明确活动类型	能够更完整地同步新增、修改、删除和移动

资源发现模块应完成以下工作：

1. 读取有序集合。
2. 根据采集策略沿页面链接遍历。
3. 读取每页中的相关活动。
4. 在可用时保留活动类型和时间。
5. 只为相关资源提取地址，尤其是 Manifest。
6. 去除重复引用，但保留来源和最新活动。
7. 保存检查点，以支持后续增量运行。

该模块汇总的是 Manifest 引用，不会把多个 Manifest JSON 合并成一个 Manifest。

5. IIIF Registry

登记目录的机器入口是：

```
https://registry.iiif.io/index.json
```

它本身是一个 `OrderedCollection`，并指向登记目录的分页。登记活动进一步指向各提供机构的发现活动流。本项目可以用它启动机构普查：

```
Registry 入口
→ Registry 分页
→ 已登记的机构活动流
→ 机构活动流分页
→ Manifest 引用
→ 当前 Manifest JSON
```

Registry 网页和 Registry JSON 的用途不同：网页供人阅读，`index.json` 才是程序入口。同样，Change Discovery 的规范页面用于解释标准，并不是可以直接采集的数据集。

6. Image API 3.0

Image API 根据标准地址返回图像，主要请求模板如下：

```
{scheme}://{server}{/prefix}/{identifier}/{region}/{size}/{rotation}/{quality}.{format}
```

示例：

```
https://example.org/image-service/abc/full/max/0/default.jpg
```

各参数在概念上依次表示区域、尺寸、旋转、质量和格式。图像服务还会在下列地址提供技术信息：

```
{scheme}://{server}{/prefix}/{identifier}/info.json
```

在本学期项目中，只有当 Manifest 通过已经确认的版权规则，并且下游确实需要图像文件时，才应下载图像。即使暂不下载，也应保存 Image API 服务地址。

7. 本项目的采集工作流程

按照当前的流水线机器设计，推荐流程如下：

```
提供机构存储与选择
→ 入口地址访问与有效性检查
→ 资源发现与分页遍历
→ Manifest 获取与结构校验
→ 版权识别、规范化与筛选
→ 元数据提取与规范化
```

→ 公共数据模式校验与导出
→ Hafidz 的下游内容提取与 Time Atlas

7.1 提供机构存储与选择

输入：Registry 及导师批准的其他提供机构来源。

保存机构名称、机构网页、发现入口、馆藏重点、估计规模、版权观察结果、状态和最近检查时间。通过明确配置选择本次处理的机构，避免把机构差异直接写死在程序分支中。

输出：提供机构记录和本次运行选中的数据入口。

7.2 入口地址访问检查

输入：一个提供机构的数据入口。

设置超时和明确的客户端标识后发出请求，记录重定向后的最终地址、HTTP 状态、响应内容类型、访问时间和失败类别。HTTP 请求成功并不代表返回内容一定是合法 IIIF JSON。

输出：访问结果，以及提供给资源发现模块的响应或已经验证的地址。

7.3 资源发现

输入：有效的 Change Discovery 活动流或 IIIF Collection。

遍历分页，识别相关资源引用，去除重复地址，并保存发现来源、活动类型和时间。Change Discovery 与 Presentation Collection 的数据结构不同，可以分别解析，但应输出统一格式。

输出：包含来源和发现状态的 Manifest 地址队列或表格。

7.4 Manifest 获取与校验

输入：Manifest 地址。

获取当前 JSON，记录 HTTP 结果，解析内容，在条件允许时识别 Presentation API 版本，并校验基本结构。缓存原始响应或计算哈希值，可以减少不必要的重复请求，并支持日后的版本比较。

输出：原始 Manifest 及校验状态；失败时输出明确的失败记录。

7.5 版权处理

输入：结构可用的 Manifest。

保存原始版权 URI 或声明，将已知声明映射为统一类别，并且只应用导师确认的允许列表。可以使用 `allowed`、`excluded`、`unknown`、`pending_policy` 等明确状态。每次排除都应记录原因。

输出：版权决定及可以审查的解释。

7.6 元数据提取与规范化

输入：通过版权规则的 Manifest。

至少提取标识符、日期、地点、版权、来源机构、Manifest 地址和发现入口。原始值与规范化值分开保存。馆藏机构所在地与对象涉及或描绘的历史地点必须区分。缺失和模糊数据应明确表示。

输出：临时规范化记录。

7.7 公共数据模式校验与导出

输入：临时规范化记录。

将记录映射到带版本号的团队公共数据模式，检查必需字段、字段类型、缺失值规则和来源追踪信息，再导出 JSONL 或 Hafidz、Kaile 确认的其他格式。

输出：符合模式的记录，以及单独保存的校验错误报告。

8. 第一版原型的最小记录

最终模式必须由团队确认。讨论时可以参考以下临时记录：

```
{
  "identifier": "https://example.org/iiif/object-1/manifest",
  "date": {
    "raw": "circa 1890",
    "normalized": "1890",
    "status": "approximate"
  },
  "place": {
    "raw": "Lausanne",
    "normalized": "Lausanne, Switzerland",
    "status": "normalized"
  },
  "rights": {
    "raw": "https://rightsstatements.org/vocab/NoC-US/1.0/",
    "category": "no_known_copyright",
    "decision": "pending_policy"
```

```
  },
  "source": {
    "provider": "Example Institution",
    "manifest_url": "https://example.org/iiif/object-1/manifest",
    "discovery_endpoint": "https://example.org/activity-stream"
  },
  "processing": {
    "retrieved_at": "2026-09-14T12:00:00Z",
    "status": "normalized",
    "schema_version": "provisional-0.1"
  }
}
```

这只是供讨论的实现建议，不是导师批准的公共数据模式或版权决定。

9. 可靠运行原则

第一目标应是让少量真实数据完整通过所有模块。当 10–100 个真实 Manifest 能够从发现阶段走到导出阶段后，再加入重试、频率限制、检查点、去重、缓存、统计和可复现配置。

每条记录都应能够追溯到提供机构、发现入口和 Manifest 地址。缺失值和失败不能被隐藏。日志至少记录来源、时间、处理阶段、结果和失败原因。

10. 简短口头说明

> IIIF 是一组帮助文化遗产机构以统一方式发布数字对象的标准。Change Discovery API 帮助我们跨机构发现 Manifest 地址。Presentation API 定义的 Manifest 描述一个对象及其页面顺序、元数据、版权和图像链接。如果对象提供图像服务，Image API 会规定获取所需图像的统一地址格式。我的项目将调查 IIIF 提供机构，发现并获取 Manifest，校验其结构，按照版权进行筛选，规范化标识符、日期、地点、版权和来源，再按照团队公共数据模式导出，交给下游内容提取程序和 Time Atlas。

阅读顺序建议

1. 先阅读本指南第 1、2 和 10 节，形成整体认识并练习口头说明。
2. 再阅读第 4、5 和 7 节，理解本项目如何发现和采集资源。
3. 阅读第 3 节，理解 Manifest 的字段和结构。
4. 只有在需要获取图像时，再深入阅读第 6 节及完整 Image API 规范。
5. 实现过程中遇到边界情况时，查阅 `official-originals` 中保存的完整英文规范。